

Fourier Analysis of Piano Chords:

Decomposing Major and Minor Consonance

To what extent can Fourier analysis be used to mathematically decompose musical chords into their individual notes, and how do the resulting frequency structures differ between major and minor chords?

lvx164

27 pages

1 Motivation and Introduction

I have been playing the piano for over ten years and have noticed a particular musical ability, which is to recognize and name pitches, even if they are played together as a chord. This ability is known as absolute pitch. While most people cannot identify individual pitches within a chord, they can usually distinguish between major and minor chords, because they evoke different emotions and feelings. Major chords are often associated with values such as "warm" or "bright", while minor chords are commonly perceived as more "melancholic".

So this led me to wonder whether it is possible to differentiate different individual notes within a chord, physically and mathematically. Moreover, whether these perceived differences between major and minor chords are related to maybe fundamental, mathematical properties of sound waves.

The goal of this investigation is to understand the mathematical nature of musical notes and chords. To be more specific, I hope to examine whether individual notes within a chord can be separated mathematically, and whether measurable differences exist in the structure between major and minor chords.

Research Question

To what extent can Fourier analysis be used to mathematically decompose musical chords into their individual notes, and how do the resulting frequency structures differ between major and minor chords?

2 The nature of notes

Firstly, I thought that in order to understand a complex chord consisting of two or more notes within a chord, I must first understand how one note would be understood in the physical world.

Sound waves are essentially pressure waves, meaning that they are mechanical vibrations that travel through a medium such as air, water, or solids to transfer energy. They are created by a vibrating object that causes particles in the medium to vibrate back and forth, which disturbs adjacent particles and creates a wave of compressions and rarefactions, as shown in Figure 1. This kinetic energy, when arriving at our ears, will then be transferred and received as sounds.

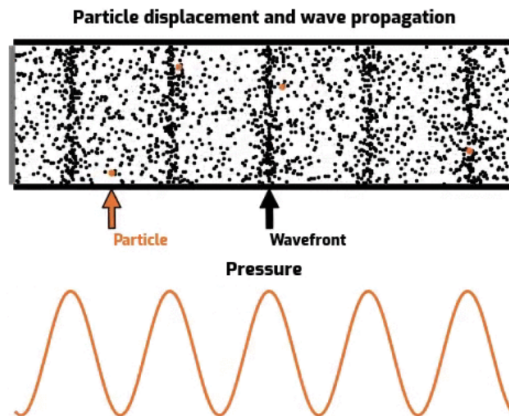


Figure 1: Wave models (“The Science Behind the Sound”)

If we consider the position of a single particle within the medium, and how that changes over time, we can model its displacement as an oscillation about its equilibrium position. Since this displacement is repetitive, it could be described mathematically by a periodic function.

Given a perfect situation where no energy is lost during transmission, we could describe a pure note using a sine wave.

$$x(t) = A \sin(\omega t)$$

I chose a sine wave because it starts at the equilibrium position. Here, A is the amplitude, which is the maximum displacement of air particles from equilibrium. Intuitively, displacement determines the amount of kinetic energy carried within this sound, which is the volume. ω is the angular frequency determining how fast the function oscillates.

For example, if we imagine a sound wave that completes one full cycle in exactly one second. In this case, the frequency of the wave is $f = 1$ Hz. Since one complete cycle of a sine wave corresponds to an angle of 2π radians, the angular frequency must be $\omega = 2\pi \times 1 = 2\pi$ rad/s. Thus, the general relationship between the two quantities is found to be

$$\omega = 2\pi f$$

Then I wanted to verify this relationship by using a real-world example of the middle C on the piano. The middle C on the piano (C_4) has a frequency of $f = 261.6$ Hz. Its corresponding angular frequency would be

$$\omega = 2\pi \times 261.6 = 523.2\pi \text{ rad/s.}$$

I decided that A should be 1, as it is just a relative volume. Subsequently, the equation

of the sound wave for C is

$$x_C(t) = A \sin(\omega t) = \sin(523.2\pi t)$$

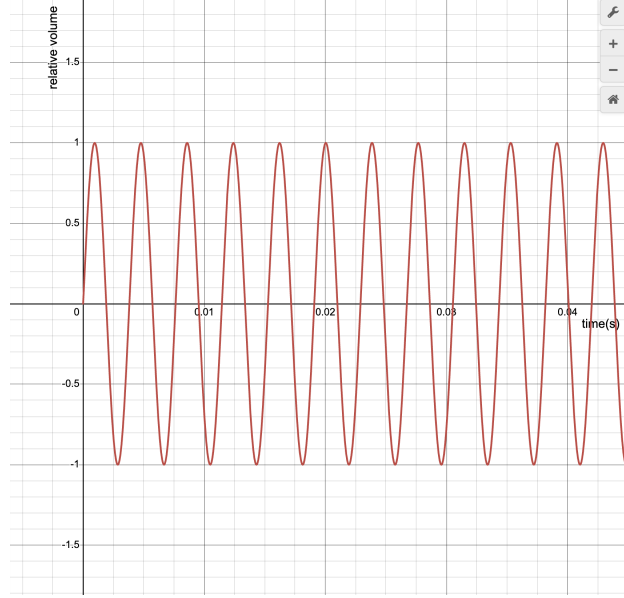


Figure 2: A pure tone (e.g., middle C) modeled as a sine wave.

Then, I verified this by taking a C_4 audio and reading it with the Audacity software.

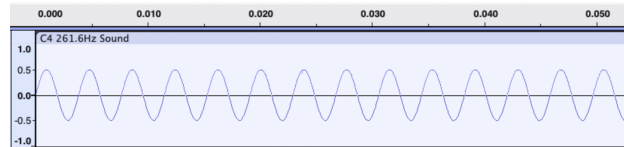


Figure 3: A pure C_4 frequency in the Audacity music manipulation software

If we count the number of peaks between $t = 0$ and $t = 0.04$ for Figure 2 and Figure 3, we arrive at the same number of 11, proving our hypothesis to be correct. The shape of the graph also looks exactly the same, apart from the Amplitude A . The real audio has an $A = 0.5$ instead of $A = 1$, but since A is just a relative value, its significance could be ignored in this scenario.

However, this represents only one note within a perfect environment; subsequently, it is also important to consider how multiple frequencies interact with each other.

3 Single frequencies

On the piano, rarely only one note is played at a time, but usually at least a few, together forming a chord. Hence, I wanted to investigate a function for that.

Mathematically speaking, when two or more waves pass through the same point in space, the total displacement is the sum of their individual displacements. This idea is known as the principle of superposition. It means that if we have two pure tones $x_1(t)$ and $x_2(t)$ as the following:

$$x_1(t) = A_1 \sin(\omega_1 t) \quad \text{and} \quad x_2(t) = A_2 \sin(\omega_2 t),$$

Then the resulting combined wave is simply

$$x_{\text{tot}}(t) = x_1(t) + x_2(t) = A_1 \sin(\omega_1 t) + A_2 \sin(\omega_2 t).$$

First, I decided to consider and verify the case where two of the same note with the same frequencies are played at the same time. For example, the C_4 .

$$x_C(t) = A \sin(\omega t) = A \sin(2\pi f t) = \sin(523.2\pi t).$$

Since the two notes are completely identical, meaning that they have the peaks and valleys occur exactly at the same time, they amplify. This is also known as constructive interference.

$$x_{\text{tot}}(t) = \sin(523.2\pi t) + \sin(523.2\pi t) = 2 \sin(523.2\pi t).$$

We see that since the two notes are completely identical, the amplitude doubles, because the crests and troughs of both waves add up, as shown in Figure 4.

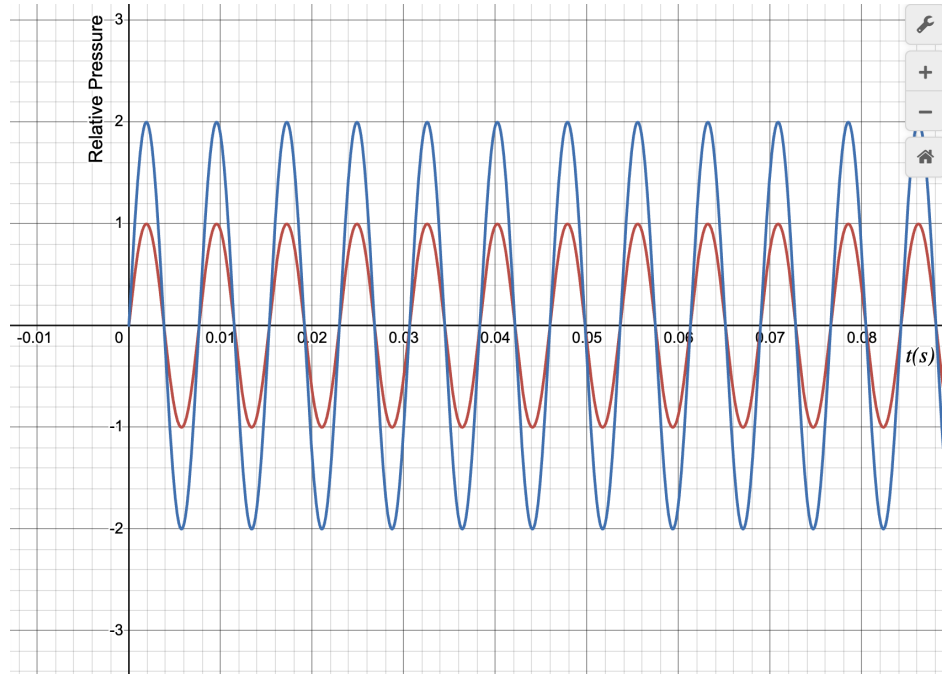


Figure 4: Graph of $\sin(523.2\pi t)$ in red and $2\sin(523.2\pi t)$ in blue

Then I wondered about a possible phase shift, ϕ , as sound waves almost never start at the exact same time.

$$x_1(t) = \sin(\omega t), \quad x_2(t) = \sin(\omega t + \phi).$$

The combined wave would then be

$$x_{\text{tot}}(t) = \sin(\omega t) + \sin(\omega t + \phi).$$

Using the trigonometric identity

$$\sin A + \sin B = 2 \sin\left(\frac{A+B}{2}\right) \cos\left(\frac{A-B}{2}\right),$$

I can simplify this to

$$x_{\text{tot}}(t) = 2 \sin\left(\omega t + \frac{\phi}{2}\right) \cos\left(\frac{\phi}{2}\right).$$

Through this equation, I can see how the amplitude depends on the phase difference ϕ :

$$A = 2 \cos\left(\frac{\phi}{2}\right).$$

A full complex-number (phasor) derivation of this result is included in Appendix A.

When $\phi = 0$, $\cos(\phi/2) = 1$, and the amplitude doubles as before (perfect constructive interference). When $\phi = \pi$, $\cos(\phi/2) = 0$, and the amplitude becomes zero, the two waves cancel completely (perfect destructive interference). Any phase difference in between causes the so-called partial interference, lowering the overall amplitude.

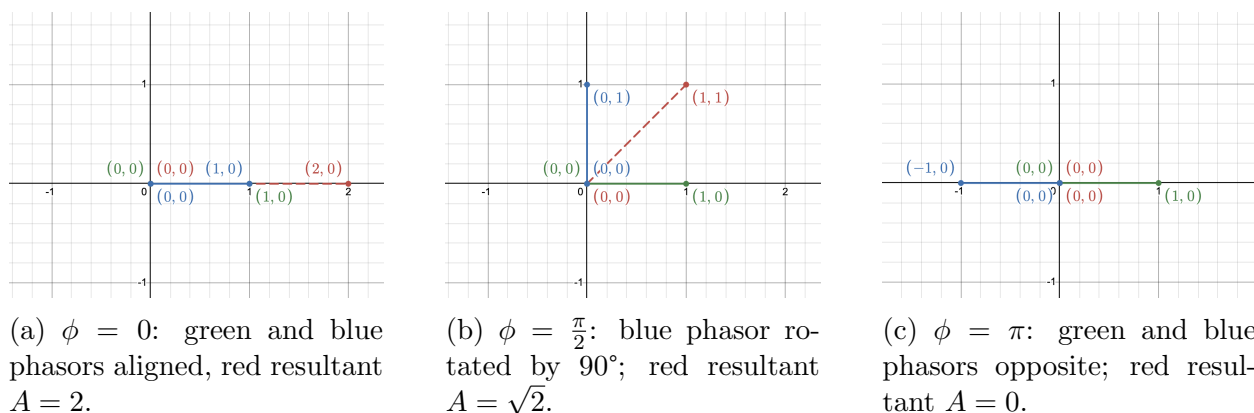


Figure 5: Vector (phasor) representation of two waves with phase difference ϕ . The green vector represents $(x_1(t))$, the blue vector represents $(x_2(t))$, and the red vector shows their resultant sum $(x_{\text{tot}}(t))$.

On the graphs above, it is clearly shown how as the angle between the phasors increases, the resultant amplitude $2\cos(\phi/2)$ decreases from construction to complete cancellation.

I thought this is rather counterintuitive, so to test and verify this idea experimentally, I simulated two identical C_4 tones in the Audacity software and introduced a small time delay between them.

Figure 6 shows the two tracks in Audacity.

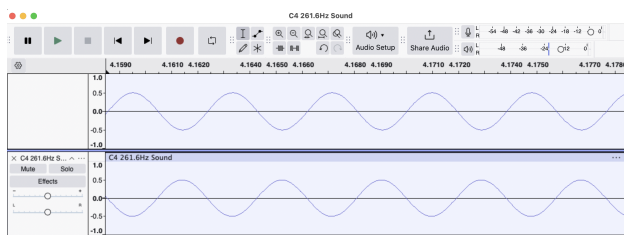
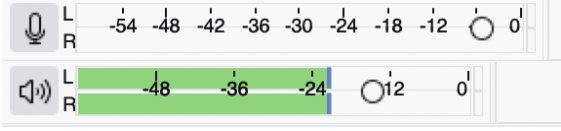


Figure 6: Two identical C_4 tones displayed in Audacity with a slight time delay between them.

When I played both tracks together, the overall volume level was noticeably lower than when the sound was played alone, confirming that partial destructive interference had occurred. This supports my hypothesis that the amplitude, which is the volume, depends on ϕ , the phase difference.



(a) Single C_4 waveform showing a consistent volume level



(b) Two C_4 tones adjusted with offset shows a reduced volume

Figure 7: Experimental verification of interference using Audacity. The overall amplitude decreases when two identical C_4 tones are slightly out of phase, consistent with the predicted reduction $2\cos(\phi/2)$ from the phasor model.

However, in reality, we don't really observe reduced volume. This is because strings are never really struck at exactly the same microsecond. Moreover, single strings produce not really a single sine wave, but rather a spectrum of overtones with slightly different frequencies and phases. As a result, the amplitude varies, which adds to the richness of the piano sound.

It turns out that actually, we can derive through physics that when N sound waves of equal amplitude but random phases are summed together, the total amplitude no longer increases linearly. The energy carried by each wave (which is proportional to A^2) adds linearly. Hence the total amplitude, when calculated, can be found approximately to be

$$A_{\text{avg}} \propto A\sqrt{N}$$

This explains why playing multiple keys on a piano makes the sound fuller but not proportionally louder. Each note introduces a complex mix of harmonics whose phases evolve independently, so the total waveform never stays perfectly aligned or canceled. Instead, the ear perceives an averaged amplitude that remains relatively stable.

4 Combined different frequencies

In practice, chords usually consist of different notes. I decide to model this by considering the case of a C_4 (261.6 Hz) and E_4 (329.6 Hz).

$$x_C(t) = \sin(2\pi f_C t) = \sin(523.2\pi t), \quad x_E(t) = \sin(2\pi f_E t) = \sin(659.2\pi t).$$

By applying the principle of superposition, the combined waveform of C_4 and E_4 becomes

$$x_{\text{tot}}(t) = x_C(t) + x_E(t) = \sin(2\pi f_C t) + \sin(2\pi f_E t) = \sin(523.2\pi t) + \sin(659.2\pi t).$$

When I plotted this new wave on Figure 9, I immediately noticed that the resulting waveform was no longer a simple sine function. Instead, the amplitude varied, with both

constructive and destructive interference patterns.

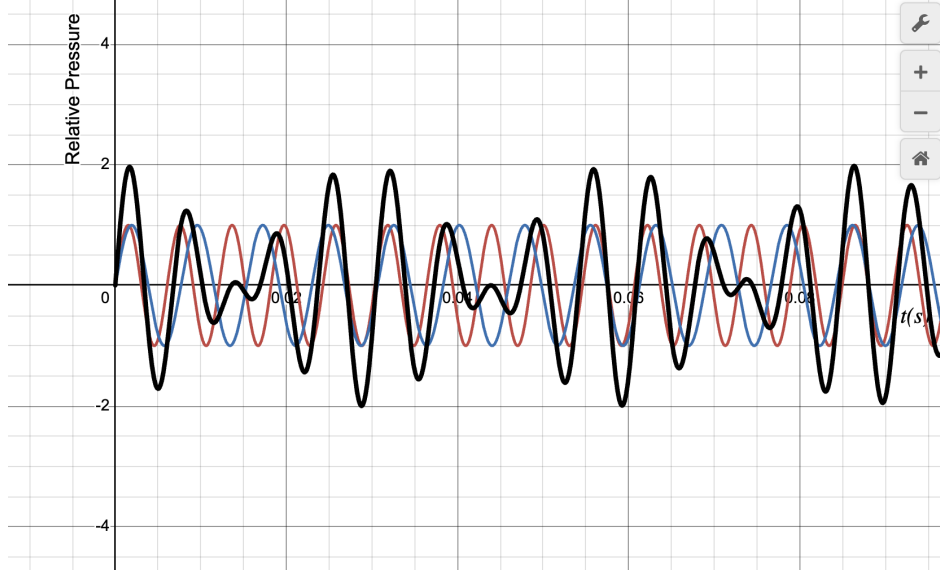


Figure 8: Graph of $x_{\text{tot}}(t) = \sin(523.2\pi t) + \sin(659.2\pi t)$ in black, representing the combined waveform of C_4 (red) and E_4 (blue).

To better understand the resulting shape, I apply the identity

$$\sin a + \sin b = 2 \sin\left(\frac{a+b}{2}\right) \cos\left(\frac{a-b}{2}\right),$$

with $a = 2\pi f_C t$, and $b = 2\pi f_E t$

$$\begin{aligned} x_{\text{tot}}(t) &= \sin(2\pi f_C t) + \sin(2\pi f_E t) \\ &= 2 \sin\left(\frac{a+b}{2}\right) \cos\left(\frac{a-b}{2}\right) \\ &= 2 \sin\left(2\pi \frac{f_C + f_E}{2} t\right) \cos\left(2\pi \frac{f_E - f_C}{2} t\right). \end{aligned}$$

Defining that

$$f_{\text{avg}} = \frac{f_C + f_E}{2}, \quad \Delta f = f_E - f_C,$$

the expression becomes

$$x_{\text{tot}}(t) = 2 \sin(2\pi f_{\text{avg}} t) \cos\left(2\pi \frac{\Delta f}{2} t\right).$$

This shows that the resulting wave has a rapid oscillation at the average frequency f_{avg} , and the amplitude is determined by the cosine term, dependent on the frequency difference

Δf .

Specifically for C_4 and E_4 , we have

$$f_{\text{avg}} = \frac{f_C + f_E}{2} = \frac{261.6 + 329.6}{2} = 295.6 \text{ Hz}, \quad \Delta f = f_E - f_C = 68.0 \text{ Hz}$$

This structure is illustrated in Figure 9.

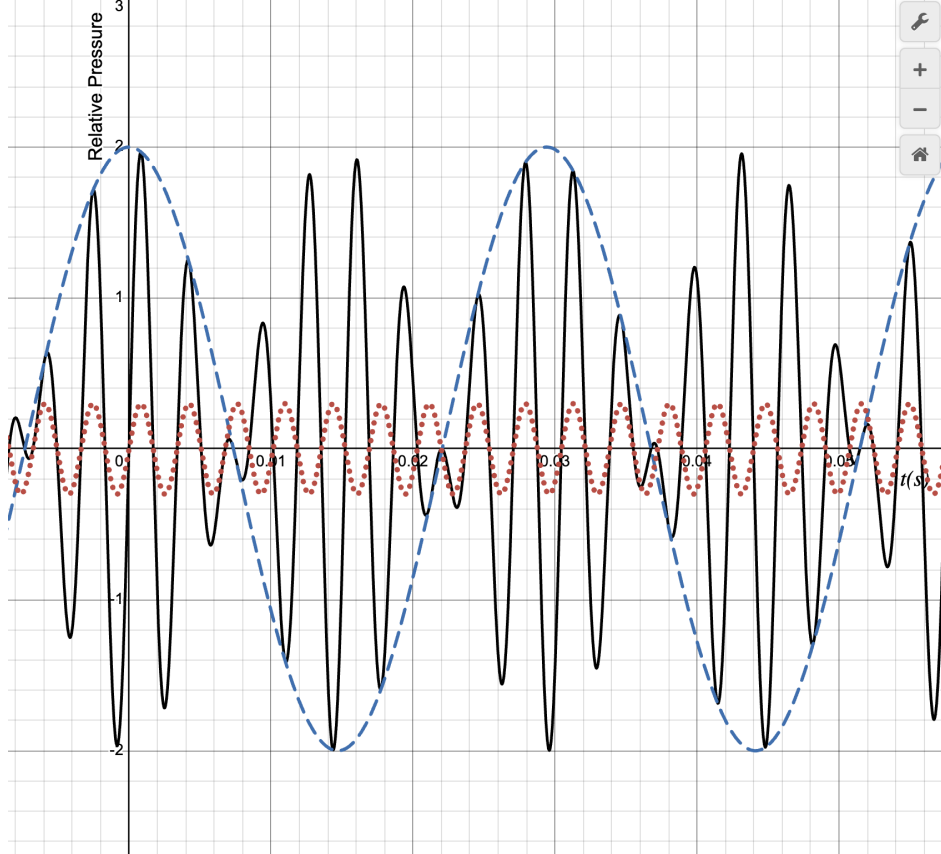


Figure 9: Combined waveform $x_{\text{tot}}(t) = \sin(523.2\pi t) + \sin(659.2\pi t)$ (black), wrapped in $2 \cos(2\pi \cdot 34.0t)$ (blue) and having the same period as $\sin(2\pi \cdot 295.6t)$ (red).

In Figure 9, we observe a periodic growth and decay of amplitude, which is called beating.

Beat Frequency

The amplitude of oscillation is determined specifically by the cosine term. The frequency of oscillation of this term is $\frac{\Delta f}{2}$. The beat frequency, in other words, the number of maxima per second, is

$$f_{\text{beat}} = |f_2 - f_1|.$$

So, for C_4 and E_4 ,

$$f_{\text{beat}} = 68.0 \text{ Hz.}$$

This basically means that the constructive interference takes place 68 times per second, and the envelope itself has a frequency of 34 full oscillations per second.

Later, I will compare these predicted frequency separations with the spacing between dominant peaks observed in the FFT spectra of recorded major and minor chords.

5 Transition to the practical

Up until here, I have explored how theoretically sound waves interact with each other. I have simulated them with sine waves, explored how amplitudes and phase change when different interferences happen. Although it explains the nature of chords and musical sounds well, it does not yet address anything about the difference between major and minor chords. Hence, I decided to switch the perspective and analyze from the viewpoint of an existing recording.

Essentially, a chord is the sum of frequency waves:

$$x(t) = \sum_{n=1}^N x_n(t) = \sum_{n=1}^N A_n \sin(\omega_n t + \phi_n),$$

where

Symbol	Meaning	Explanation / Unit
$x_n(t)$	Waveform (continuous)	Instantaneous air-pressure variation
A_n	Amplitude coefficients	Strength of sinusoidal components (same unit as $x(t)$)
ω_n	Component angular frequencies	$2\pi f_n$ (rad/s)
ϕ_n	Phase shift	Relative onset of the n th component (rad)

Table 1: Summary of variables and quantities used

However, a chord is not just three fundamental frequencies played at the same time. Each key excites a whole stack of partials near integer multiples of its fundamental. This is due to the resonant nature of frequencies and sound waves. Hence, how "consonant" each type of chord, either major or minor, is, becomes a question of whether partials from different notes land near each other (less beating, more "calm"), or do they keep missing and rubbing (more beating, more "rough"). I thought I could try to investigate this using frequencies.

In order to separate the individual frequencies from the overall $x(t)$, creating the frequency graph, I decided to approach it using the Fourier Transform, more specifically, the Discrete Fourier Transform.

Symbol	Meaning	Explanation / Unit
$x_n(t)$	Waveform (continuous)	Instantaneous air-pressure variation
A_n	Amplitude coefficients	Strength of sinusoidal components (same unit as $x(t)$)
ω_n	Component angular frequencies	$2\pi f_n$ (rad/s)
ϕ_n	Phase shift	Relative onset of the n th component (rad)
T	Fundamental period	Duration of one full cycle (s)
f_0	Fundamental frequency	$1/T$ (Hz)
f_s	Sampling frequency	Samples per second, $f_s = 1/\Delta t$ (Hz)
Δt	Sampling interval	Time between samples, $\Delta t = 1/f_s$ (s)
t_n	Sample times	$t_n = n\Delta t$ (s)
x_n	Sampled amplitudes	Value of $x(t)$ at t_n
N	Number of samples	Length of the discrete record
k	Frequency-bin index	$0, 1, \dots, N-1$
X_k	DFT coefficient	Complex spectrum at index k
$ X_k $	Spectral magnitude	$ X_k = \sqrt{(\text{Real}X_k)^2 + (\text{Imaginary}X_k)^2}$
$F(\omega)$	Fourier transform	$F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-i\omega t} dt$
f_C, f_E, f_G, f_{Eb}	Pitch frequencies	Fundamental frequencies of specific notes (Hz)

Table 2: Summary of all variables and quantities used throughout

6 Experiment and Analysis

6.1 Fourier Series

The Fourier Transform is build upon the idea of the Fourier Series, which states that any periodic function $x(t)$ which satisfies

$$x(t + T) = x(t), \quad \forall t,$$

with fundamental angular frequency

$$\omega_0 = \frac{2\pi}{T} = 2\pi f_0,$$

according to the Fourier Theorem, can be expressed as a sum of sines and cosines:

$$x(t) = \sum_{n=1}^{\infty} [a_n \sin(n\omega_0 t) + b_n \cos(n\omega_0 t)]$$

The coefficients a_n and b_n represent the volume of each note. This Fourier series perspective states that any periodic function, such as the one expressing a chord triad, could be treated

as a structured collection of sinusoids even when the waveform looks complex.

6.2 Fourier Transform

However, real recordings, like the ones I took, are not strictly periodic: Notes decay, and the waveform may include transients and noise. Extending the Fourier idea to non-periodic signals leads to the continuous Fourier Transform of any waveform function $f(t)$:

$$F(\omega) = \int_{-\infty}^{\infty} f(t) e^{-i\omega t} dt$$

which produces a continuous spectrum indicating the contribution of each angular frequency ω . The original signal, $f(t)$, can be recovered via the inverse transform

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) e^{i\omega t} d\omega,$$

so the transform/inverse pair preserves information while converting between time and frequency viewpoints. But, this Integral, as I noticed, is continuous. However, my recording, since it is a digital audio, is discrete and stores only finitely many samples, neither of which satisfy the bound given by the Fourier transform, hence I had to apply the Discrete Fourier Transform.

6.3 Discrete Fourier Transform (DFT)

For a Discrete set of data, suppose that the continuous waveform $x(t)$ is sampled every Δt seconds, it produces N discrete values $x_n = x(t)(t_n)$ with $t_n = n\Delta t$. Evaluating the continuous transform at $\omega_k = 2\pi k/(N\Delta t)$ and approximating the integral gives

$$F(\omega_k) \approx \sum_{n=0}^{N-1} x_n e^{-i\omega_k t_n} \Delta t = \Delta t \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}.$$

And if we absorb the constant factor Δt into the definition of the discrete spectrum, I arrive at the definition for a Discrete Fourier Transform (DFT):

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}, \quad k = 0, 1, \dots, N-1$$

Each X_k corresponds to the frequency $f_k = kf_s/N$, where $f_s = 1/\Delta t$ is the sampling rate, and encodes both magnitude and phase. The magnitude of each bin is given by

$$|X_k| = \sqrt{[\text{Real}(X_k)]^2 + [\text{Imaginary}(X_k)]^2}$$

and indicates how strongly that frequency contributes to the sampled waveform.

Sampling Resolution and Nyquist Limit

My recordings used for manipulations are digital, hence the signal is sampled at a fixed rate f_s . This is very important, because it has two limitations: the frequency resolution and the maximum detectable frequency.

The frequency resolution of the DFT is given by:

$$\Delta f = \frac{f_s}{N},$$

where N is the number of samples. This basically states by how much we can divide the frequency axis. As N increases, more samples are being taken, and frequencies can be separated more precisely.

Another limitation of the DFT is stated by the Nyquist–Shannon sampling theorem, which claims that the highest frequency that can be correctly represented is

$$f_{\max} = \frac{f_s}{2}.$$

Any frequencies above this limit would appear incorrectly as lower frequencies, which means that are aliased. Luckily, piano fundamentals lie below 500 Hz, and my f_s is large enough, being 42300 Hz. Hence, no aliasing should take place in my sample.

6.4 Manual Calculation Example

Using the following data, a manual Fourier transform could be attempted. I took the first 23 data points from the C major chord, which I recorded using Audacity and my piano. These data points cover the first 0.000520 milliseconds of my entire recording. I recorded at $f_s = 42300$ Hz.



Figure 10: Graph of recording from C major

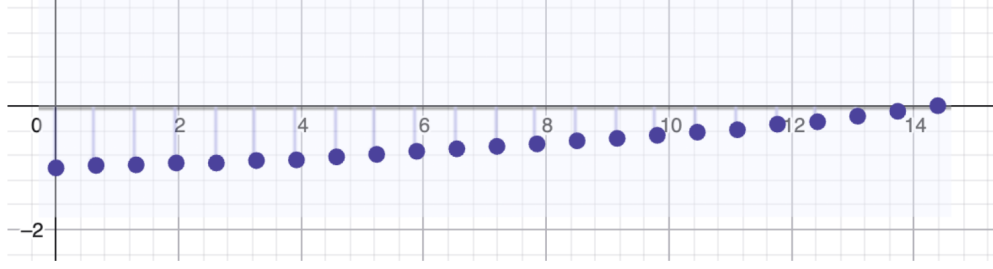


Figure 11: graph of the first 23 datapoints of C major recording in Geogebra

I then rescaled my audio wave graph, and received the following data:

n	t_n (ms)	x_n (amplitude)
1	0.000000	-1.00
2	0.653877	-0.96
3	1.307754	-0.95
4	1.961630	-0.92
5	2.615507	-0.92
6	3.269384	-0.88
7	3.923261	-0.87
8	4.577138	-0.82
9	5.231015	-0.78
10	5.884891	-0.73
11	6.538768	-0.69
12	7.192645	-0.65
13	7.846522	-0.61
14	8.500399	-0.56
15	9.154275	-0.52
16	9.808152	-0.47
17	10.462029	-0.42
18	11.115906	-0.38
19	11.769783	-0.29
20	12.423660	-0.25
21	13.077536	-0.16
22	13.731413	-0.08
23	14.385290	0.01

Table 3: Recorded amplitudes x_n at times $t_n = n\Delta t$ for a C-major chord (values rounded to three decimals).

Then I tried to manually compute the DFT using my datapoints, shown below

$$X_0 = \sum_{n=0}^{22} x_n = -13.9, \quad |X_0| = 13.9$$

$$X_1 = \sum_{n=0}^{22} x_n e^{-\frac{2\pi i(1)n}{23}} = \sum_{n=0}^{22} x_n \left(\cos \frac{2\pi n}{23} - i \sin \frac{2\pi n}{23} \right) = 0.277 + 3.762i, \quad |X_1| = 3.772$$

$$X_2 = \sum_{n=0}^{22} x_n e^{-\frac{2\pi i(2)n}{23}} = \sum_{n=0}^{22} x_n \left(\cos \frac{4\pi n}{23} - i \sin \frac{4\pi n}{23} \right) = -0.174 + 1.893i, \quad |X_2| = 1.901$$

$$X_3 = \sum_{n=0}^{22} x_n e^{-\frac{2\pi i(3)n}{23}} = \sum_{n=0}^{22} x_n \left(\cos \frac{6\pi n}{23} - i \sin \frac{6\pi n}{23} \right) = -0.420 + 1.201i, \quad |X_3| = 1.272$$

...

I soon noticed that even 23 data points, only the first 0.000520 milliseconds of my entire recording, were too large for me to compute manually, so I decided to approach this using a Python program with FFT.

6.5 Fast Fourier Transform (FFT)

DFT has a very large complexity $\mathcal{O}(N^2)$. FFT, on the other hand, while having the very same mathematical concept, would reduce the complexity to $\mathcal{O}(N \log N)$ (“Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT)”).

So I decided to perform FFT on C- and D-major, minor.

7 Hypotheses

In idealized conditions (with no interference and perfect notes), major triads, such as C-major, have frequency ratios of 4 : 5 : 6, and minor chords 10 : 12 : 15. Due to the law of superposition, more of the harmonics (peaks and valleys) of the separate tones in a major chord will coincide than in a minor chord. This is because smaller integer ratios share more common multiples. This means that the outcome would be more “smooth” in terms of the number of peaks and valleys, while minor triads have a more “chaotic” alignment pattern, increasing the roughness of the distribution.

So there are two simple hypotheses that I look for:

- Major triads follow 4 : 5 : 6, so many low-order partials coincide or sit very close, so the spectrum shows fewer unmatched partials.

- Minor triads follow 10 : 12 : 15, so I expect more near-misses between partials, meaning broader or doublet peaks.

To evaluate this quantitatively, I wanted to define an alignment metric.

According to Plomp & Levelt 1965, when partials lie close but do not coincide, the interference pattern produces beating, which is directly linked to the perceptual roughness of a chord. On the other hand, when partials coincide, the energy is reinforced and produces a more stable structure.

I will consider the first H partials of a chord with fundamental frequencies f_i , for $i \in \{1, 2, 3\}$:

$$P_i = \{nf_i \mid n = 1, 2, \dots, H\}.$$

And a partial $p \in P_i$ is considered to be aligned if there is another partial $q \in P_j$ (with $j \neq i$) such that

$$|p - q| \leq \tau,$$

where τ is a tolerance window, taking FFT resolution, piano tuning, etc., into account.

Let M be the total number of aligned partials across all three chord tones, and let $N = 3H$ be the total number of partials considered. The harmonic alignment score is defined as

$$S = \frac{M}{N}, \quad 0 \leq S \leq 1.$$

As S increases, there is a stronger harmonic coincidence between the tones within the chord, hence sounding more "smooth".

I will use $H = 6$ and $\tau = 10$ Hz in this investigation, because from my tests, these are the most reasonable values. And based on integer frequency ratios (e.g. 4:5:6 for major versus 10:12:15 for minor), I hypothesize that

$$S_{\text{major}} > S_{\text{minor}}.$$

8 Results

Below are the spectra I obtained. Peaks align with the three chord tones, and the surrounding structure shows how the partials group.

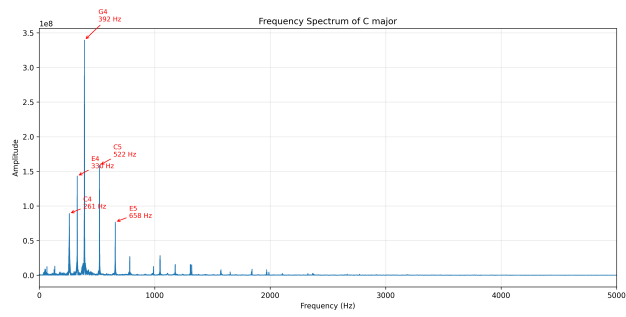


Figure 12: C major

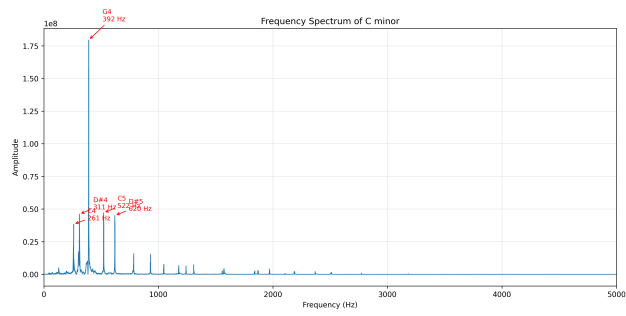


Figure 13: C minor

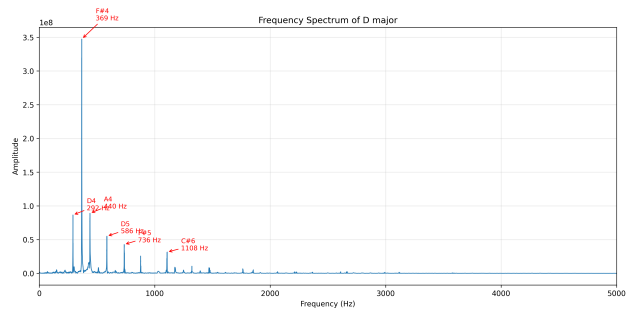


Figure 14: D major

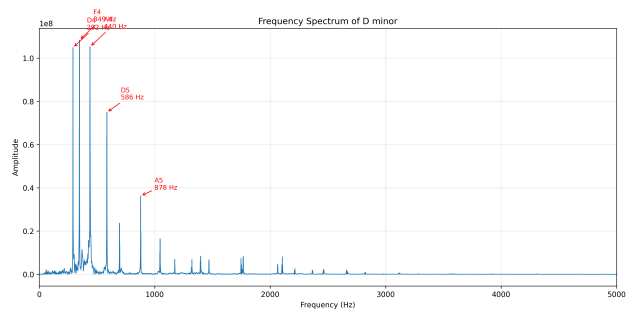


Figure 15: D minor

Tables 4–7 list the ten strongest unique spectral peaks (below 5 kHz) detected for each chord.

Rank	f (Hz)	Note	Detune (cents)	Amplitude
1	67.04	C2	+42.72	12 370 413
2	135.08	C#3	-44.45	13 021 019
3	260.68	C4	-6.27	88 995 828
4	268.81	C4	+46.90	11 122 718
5	329.52	E4	-0.59	143 156 336
6	370.87	F#4	+4.08	12 259 467
7	380.94	G4	-49.51	17 049 549
8	392.02	G4	+0.10	339 364 977
9	400.15	G4	+35.64	16 088 749
10	521.66	C5	-5.28	158 633 017

Table 4: C major: strongest unique FFT peaks (≤ 5 kHz).

Rank	f (Hz)	Note	Detune (cents)	Amplitude
1	260.74	C4	-5.86	38 359 723
2	299.50	D4	+34.07	6 654 694
3	310.74	D#4	-2.15	46 139 164
4	370.82	F#4	+3.88	8 613 519
5	381.32	G4	-47.80	10 181 386
6	392.06	G4	+0.30	179 449 878
7	400.49	G4	+37.13	8 865 565
8	522.39	C5	-2.84	47 168 059
9	620.24	D#5	-5.60	45 053 611
10	785.28	G5	+2.85	15 849 320

Table 5: C minor: strongest unique FFT peaks (≤ 5 kHz).

Rank	f (Hz)	Note	Detune (cents)	Amplitude
1	292.23	D4	-8.48	86 517 780
2	368.76	F#4	-5.80	347 364 967
3	376.94	F#4	+32.19	13 997 209
4	430.41	A4	-38.15	16 196 482
5	439.83	A4	-0.66	89 072 193
6	586.28	D5	-3.11	55 066 168
7	735.86	F#5	-9.68	42 754 931
8	878.18	A5	-3.59	25 633 014
9	1108.17	C#6	-0.87	31 542 958
10	1322.39	E6	+5.08	10 576 668

Table 6: D major: strongest unique FFT peaks (≤ 5 kHz).

Rank	f (Hz)	Note	Detune (cents)	Amplitude
1	292.23	D4	-8.48	104 858 928
2	302.06	D4	+48.82	9 205 233
3	312.15	D#4	+5.66	4 858 170
4	325.37	E4	-22.51	4 404 871
5	338.34	E4	+45.19	5 977 854
6	348.59	F4	-3.15	108 378 382
7	370.82	F#4	+3.88	11 495 037
8	379.91	F#4	+45.81	5 501 792
9	389.83	G4	-9.58	6 578 812
10	400.41	G4	+36.77	5 782 604

Table 7: D minor: strongest unique FFT peaks (≤ 5 kHz).

Across all chords, the largest peaks aligned with the fundamental notes: C major/minor show C_4 (≈ 261 Hz) and G_4 (≈ 392 Hz), with the third separating them (E_4 at ≈ 330 Hz versus $E\flat_4$ at ≈ 311 Hz). Similarly, D major shows D_4 (≈ 292 Hz), $F\sharp_4$ (≈ 369 Hz), and A_4 (≈ 440 Hz), while D minor replaces the third with F_4 (≈ 349 Hz). This confirms that FFT can decompose chords into their component notes.

Now I can use the harmonic alignment index, which is defined as $S = \frac{M}{N}$. I have $N = 3H = 18$, because I consider $H = 6$ harmonics for each of the three chord tones.

Two partials are considered aligned if

$$|p - q| \leq 10 \text{ Hz.}$$

For example, for the C major, the detected fundamentals from Table 4 are:

$$f_C = 260.68 \text{ Hz}, \quad f_E = 329.52 \text{ Hz}, \quad f_G = 392.02 \text{ Hz}.$$

The first six harmonics of C are:

$$260.68, 521.36, 782.04, 1042.72, 1303.40, 1564.08.$$

The first six harmonics of G are:

$$392.02, 784.04, 1176.06, 1568.08, 1960.10, 2352.12.$$

Because

$$|3f_C - 2f_G| = |782.04 - 784.04| = 2.00 \text{ Hz} \leq 10 \text{ Hz}$$

$$|6f_C - 4f_G| = |1564.08 - 1568.08| = 4.00 \text{ Hz} \leq 10 \text{ Hz}$$

hence we can see that these two partial pairs do satisfy the alignment condition. And no other harmonic pair is within this window.

Therefore:

$$M = 2, \quad S = \frac{2}{18} = 0.11.$$

Applying the same procedure to each chord gives:

Chord	M	S
C major	2	0.11
C minor	1	0.06
D major	2	0.11
D minor	1	0.06

Overall, in both C and D, the major chords produce two alignments, while the minor chords only produce one. Hence, it does support the hypothesis of

$$S_{\text{major}} > S_{\text{minor}}.$$

9 Conclusion and Evaluation

Through this investigation, I wanted to determine to what extent Fourier analysis and wave superposition can distinguish between major and minor piano chords.

I started with mathematical models of sine waves and superposition. Through analyzing the phase and frequency differences, I derived how amplitudes are regulated, and how beating

is affected by the pitch of the note selected. Then I applied the Discrete Fourier Transform to my own recorded chords, which allowed me to decompose the waves into single frequencies within the chord. The resultant spectrum, then again, confirms that both major and minor chords consist of sets of harmonics, and due to instabilities in interference, fluctuations happen in both major and minor chords.

Then I wanted to move away from purely qualitative observations, hence I introduced the harmonic alignment index metrics. Through this, I wanted to quantify the partial alignment within a chord. Based on the example of C major and minor, major chords indeed had a higher alignment, with 11%, than minor chords, with 6%. While this is not a significant number, it does positively suggest that this Fourier analysis can distinguish major and minor chords. Again, this is supported by the previous theoretical analysis and expectation.

I suspected the relatively small observed difference between major and minor chords had several reasons.

The reason my experiment was less successful has several reasons. The recordings were made in a rather loud environment, introducing room resonance and background noise. Manually pressing the piano keys also introduced inconsistency in amplitude and timing. Furthermore, the piano itself is not perfectly harmonic, its string stiffness slightly detunes higher partials—so perfect integer ratios could not be expected.

Firstly, the piano may be slightly out of tune, which means intervals do not exactly match the ideal ratios $4 : 5 : 6$ or $10 : 12 : 15$, reducing the expected harmonic coincidences even in major chords.

Second, there are resolution limits associated with using digital tools for analysis. The FFT frequency resolution is $\Delta f = \frac{f_s}{N}$, meaning nearby partials may fall into adjacent bins or merge, reducing the apparent distinction between chord types.

Third, real piano strings exhibit inharmonicity due to string stiffness. Instead of exact integer multiples, higher partials follow $f_n = nf_0\sqrt{1 + Bn^2}$, where B is the inharmonicity coefficient. As n increases, partials deviate further from ideal harmonic positions, weakening exact alignment between chord tones. This naturally reduces the contrast predicted by simple integer models.

Other possible sources of inaccuracies include room resonance, background noise, and inconsistencies in manually pressing the keys.

Despite these limitations, my quantitative comparison of harmonic mismatch rates does support the hypothesis that major triads are of stronger partial alignment than minor triads. While the difference is only moderate, it is consistent with the analysis and expansion of the theoretical expectations based on the analysis of frequency ratios and wave superposition.

Overall my investigation demonstrates that by using Fourier analysis, combined with analytical metrics, it is possible to distinguish individual notes from chords, and between

major and minor chords.

Citations

- Age of Awareness. “The Science Behind the Sound.” Medium, <https://medium.com/age-of-awareness/the-science-behind-the-sound-10bdc94ad70>. Accessed 25 Feb. 2026.
- Ac Studio. “Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT) (Signals and Systems Review Part6).” Medium, 10 Nov. 2025, <https://medium.com/@acamyproducingstudio/discrete-fourier-transform-dft-and-fast-fourier-transform-fft-signals-and-systems-review-4765a04fb436>. Accessed 25 Feb. 2026

Consultations

- 3blue1brown. 3blue1brown (YouTube Channel), YouTube, <https://www.youtube.com/c/3blue1brown>. Accessed 25 Feb. 2026.
- Strauss, Glenn. CSE 41: Introduction to Computing Systems. Massachusetts Institute of Technology, <https://math.mit.edu/~gs/cse/websections/cse41.pdf>. Accessed 25 Feb. 2026.
- Belk, James. Fourier Series. Cornell University, <https://e.math.cornell.edu/people/belk/measuretheor>. Accessed 25 Feb. 2026.
- Chong, Y. D. Fourier Transform Lecture Notes. Nanyang Technological University, https://www1.spms.ntu.edu.sg/~ydchong/teaching/10_fourier_transform.pdf. Accessed 25 Feb. 2026.
- Lesson 17 — Fourier Transform. University College London, <https://www.ucl.ac.uk/~ucahmdl/LessonPlans/Lesson17.pdf>. Accessed 25 Feb. 2026.

Appendix

A Complex-number derivation of phase-dependent amplitude

Another more intuitive approach to justify the phase-dependent amplitude relationship is through representing the waves using as rotating vectors in the complex planes.

$$x_1(t) = e^{i\omega t}, \quad x_2(t) = e^{i(\omega t + \phi)}$$

When I add them, it gives

$$x_{\text{tot}}(t) = e^{i\omega t} (1 + e^{i\phi}).$$

The amplitude can be represented by

$$|x_{\text{tot}}| = |e^{i\omega t}| |1 + e^{i\phi}|$$

Since $|e^{i\omega t}| = 1$, the amplitude is given by

$$|x_{\text{tot}}| = |1 + e^{i\phi}|$$

We can expand $e^{i\phi}$ using Euler's formula $e^{i\phi} = \cos \phi + i \sin \phi$, and obtain

$$|x_{\text{tot}}| = |(1 + \cos \phi) + i \sin \phi|$$

The magnitude of a complex number $a + bi$ is defined to be $\sqrt{a^2 + b^2}$, hence we continue expanding

$$|x_{\text{tot}}| = \sqrt{(1 + \cos \phi)^2 + (\sin \phi)^2} = \sqrt{2 + 2 \cos \phi}$$

Using the identity $\cos \phi = 2 \cos^2(\frac{\phi}{2}) - 1$, we obtain

$$|x_{\text{tot}}| = \sqrt{4 \cos^2\left(\frac{\phi}{2}\right)} = 2 \left| \cos\left(\frac{\phi}{2}\right) \right|$$

This confirms the phase-dependent amplitude relationship derived in the main body.

B Python Program for FFT Analysis

Listing 1: Python script used for FFT analysis of piano chords.

```
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile

# ----- helpers -----
NOTE_NAMES = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
def hz_to_note(freq):
```

```

    if freq <= 0:
        return (" ", " ", 0.0)
    midi = 69 + 12 * np.log2(freq / 440.0)    # A4=440 -> MIDI 69
    m = int(round(midi))
    name = NOTE_NAMES[m % 12]
    octave = m // 12 - 1
    cents = (midi - m) * 100
    return name, octave, cents

piece_label = "C major"    # <-- use this ONLY for filenames/titles

# ----- Read audio -----
fs, data = wavfile.read("full/" + piece_label + ".wav")
if data.ndim > 1:
    data = data[:, 0]
data = data.astype(float)

# optional: reduce leakage & DC
data -= data.mean()
data *= np.hanning(len(data))

# ----- Fourier Transform -----
N = len(data)
fft_data = np.fft.fft(data)
freqs = np.fft.fftfreq(N, 1/fs)
magnitude = np.abs(fft_data)

# positive half only
half_N = N // 2
freqs = freqs[:half_N]
magnitude = magnitude[:half_N]

# ----- Pick unique peaks (no SciPy) -----
K = 8
df = freqs[1] - freqs[0]
tol_hz = 12.0
tol_bins = max(1, int(tol_hz / df))

cand = np.argsort(magnitude)[-200:]    # top 200 bins by
    ↪ amplitude
cand = cand[np.argsort(magnitude[cand])[:, -1]]    # loudest first
unique_idx = []
for idx in cand:
    if all(abs(idx - u) > tol_bins for u in unique_idx):
        unique_idx.append(idx)

```

```

        if len(unique_idx) == K:
            break
unique_idx.sort(key=lambda i: freqs[i])

# ----- Plot -----
fig = plt.figure(figsize=(12, 6))
plt.plot(freqs, magnitude, linewidth=1)
plt.title(f"Frequency Spectrum of {piece_label}")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.xlim(0, 5000)
plt.grid(True, alpha=0.3)

for i in unique_idx:
    f = freqs[i]
    amp = magnitude[i]
    note_name, octv, cents = hz_to_note(f)
    label = f"{note_name}{octv}\n{int(round(f))} Hz"
    plt.annotate(label,
                 xy=(f, amp),
                 xytext=(f + 120, amp * 1.08),
                 arrowprops=dict(arrowstyle="->", color='red'),
                 fontsize=9, color='red')

plt.tight_layout()

# ensure output folder and save
os.makedirs("figs", exist_ok=True)
fig.savefig(f"figs/{piece_label}.png", dpi=300, bbox_inches='tight')
plt.show()

# ----- Console table for IA -----
rows = []
for rank, i in enumerate(unique_idx, start=1):
    f = freqs[i]; amp = magnitude[i]
    note_name, octv, cents = hz_to_note(f)
    rows.append((rank, f, note_name, octv, cents, amp))

print("\nUnique spectral peaks ( 5 kHz):")
print("Rank  Frequency(Hz)  Note  Oct  Detune(cents)  Amplitude")
for r in rows:
    print(f"{r[0]:>4}  {r[1]:>12.2f}  {r[2]:<3}  {r[3]:>2}  {r[4]:>+8.2}
    ↪ f}  {r[5]:.0f}")

# Detected chord notes (pitch classes; keep fundamentals region)

```

```
pitch_classes = []
for _, f, note_name, *_ in rows:
    if f <= 1200 and note_name and note_name not in pitch_classes:
        pitch_classes.append(note_name)
if pitch_classes:
    print("\nDetected chord notes:", ", ".join(pitch_classes))
```